

Addressing Modes, CPU Organization, and Bus Structures

Lecture Notes — Week 5

PCC CS-401: Computer Organization and Architecture

August 7, 2026

5.1 Introduction

In Week 4 we described an instruction only on paper: it has an opcode field and one or more address/operand fields, and the CPU steps through fetch, decode, and execute using the registers already in play — PC (the address of the next instruction), IR (the instruction currently executing), MAR/MDR (the address and data lines to memory), and either an accumulator AC or a register file $R0 \dots Rn-1$ plus a stack pointer SP. What we never said is *how* an instruction's address field turns into the actual location of its operand, or what hardware inside the CPU physically carries an operand from one register to another. Those two gaps are this chapter's subject, and they turn out to be one thread rather than two: addressing modes tell the CPU *where* an operand lives, and CPU organization is about *what moves it there*.

Throughout this chapter we trace a single running example, the instruction $R1 \leftarrow R2 + R3$, through every hardware design we build, so that words like “faster” and “more expensive” become concrete cycle counts and wire counts rather than adjectives.

5.2 Addressing Modes

5.2.1 Motivation: Why a Compiler Needs More Than an Opcode

Consider a compiler translating one line of C inside a loop: $A[i] = A[i] + 1$; . The instruction it generates looks something like `ADD R1, X`, but the instruction format alone never says where X actually lives. It could be a literal constant, a fixed memory cell, a pointer, or an array slot whose address changes on every iteration. Something beyond the opcode has to specify which of these it is — and that something is the **addressing mode** encoded in the instruction.

Definition (Addressing Mode). *An addressing mode is the rule, specified by the instruction, for computing the **effective address** (EA) — the actual memory location or register holding the operand — from the instruction's address field.*

Different modes trade **flexibility** (support for pointers, arrays, relative jumps) against **cost** (extra memory or bus reads needed to resolve EA). The seven modes below follow Hamacher's standard treatment of addressing modes (Hamacher, *Computer Organization*, Ch. 7) [1]; each is motivated by a code idiom a programmer already recognizes, not memorized as an isolated list.

5.2.2 Immediate Addressing

`ADD R1, #5` adds the literal value 5 to R1. The address field *is* the operand, so no EA computation and no memory access are needed to fetch it — this is the fastest possible mode, at zero extra bus or memory cycles, but the operand must be known at compile time and must fit in the field width. Compilers reach for it for loop increments, array bounds, and other small constants.

5.2.3 Direct (Absolute) Addressing

ADD R1, 1000 adds the contents of memory location 1000 to R1: here $EA = A$, where A is the address field itself, so one memory read is needed to fetch $M[EA]$. This is simple, but the address is baked permanently into the instruction — there is no relocation and no way to reach an array element without a different mode. It is the natural choice for statically allocated global variables.

5.2.4 Indirect Addressing

ADD R1, (1000) says that $M[1000]$ does not hold the operand itself but *another address*; the CPU must go there to find the real operand. Concretely, if the instruction's address field is $A = 300$, and $M[300] = 500$, then the pointer stored at location 300 leads to $EA = 500$, and the operand 42 is found at $M[500]$. This costs *two* memory reads — one to fetch the pointer, one to fetch the operand it points to — which is exactly double the cost of direct addressing, and is the price of following a pointer stored in memory.

5.2.5 Register and Register-Indirect Addressing

These are the same idea as direct and indirect addressing, but with the pointer's home moved from memory into a register, which is cheaper because registers do not require a bus transaction to read. In **register addressing**, ADD R1, R2, the operand is already sitting in a register, so there is no memory access at all — this is the cheapest mode after immediate, and is how a compiler represents a local variable. In **register-indirect addressing**, ADD R1, (R2), the register itself holds an address rather than a value, so $EA = [R2]$ and one memory read is needed to fetch $M[EA]$. Register-indirect addressing is indirect addressing's cheaper cousin: because the pointer lives in a register instead of memory, it costs one memory read instead of two.

5.2.6 Displacement Addressing — How Arrays Actually Work

The displacement family (relative, based, indexed addressing) computes $EA = [\text{register}] + \text{displacement}$. This is precisely how array indexing is compiled: a base register R_{base} holds the address of $A[0]$, an index register R_{index} holds the loop variable i , and dedicated adder hardware computes $EA = [R_{\text{base}}] + [R_{\text{index}}] = \&A[i]$ every time the instruction executes. Because EA is recomputed from a register on every execution, the very same instruction correctly accesses a different array element on every loop iteration — which is exactly why direct addressing cannot serve this role: its address field is fixed once, at compile time, while $A[i]$'s address changes every iteration. Displacement addressing is the hardware primitive underneath every array access and struct-field access in every high-level language.

5.2.7 Autoincrement / Autodecrement Addressing

ADD R1, (R2)+ uses $[R2]$ as EA and then, as a side effect of the instruction, increments R2 by the operand's size. This is register-indirect addressing plus an automatic ± 1 (or $\pm$ operand size) on the register: autoincrement steps forward, useful for array traversal or a stack pop, while autodecrement steps backward, useful for a stack push. Because the increment happens as a side effect, no separate increment instruction is needed — saving one instruction per loop iteration.

5.2.8 Addressing Modes Side by Side

Mode	Syntax	EA	Typical use
Immediate	#5	— (value in instr.)	Constants
Direct	1000	A	Global variables
Indirect	(1000)	$M[A]$	Pointers via memory
Register	R2	— (value in reg.)	Local variables
Register-indirect	(R2)	$[R2]$	Pointer variables
Displacement	d(R2)	$[R2] + d$	Array elements, structs
Autoincrement	(R2)+	$[R2]$, then $R2 += n$	Array streaming, stack pop

Every row in this table exists because some code idiom needs it.

Example 5.2.1 (Computing the Effective Address). Take the instruction `LOAD R1, 20(R2)`, using displacement addressing, with $[R2] = 1000$.

1. The mode is displacement, so $EA = [R2] + d$.
2. The instruction's displacement field gives $d = 20$.
3. $EA = 1000 + 20 = 1020$.
4. The operand is fetched from $M[1020]$ and the result is placed in $R1$.

Cost: one register read (to obtain $[R2]$) plus one memory read (to obtain $M[EA]$) — cheaper than indirect addressing's two memory reads, because one of the two accesses here is a register read rather than a memory read.

5.3 CPU Organization and Register-Transfer Notation

Addressing modes answer *where* an operand lives. The next question is *how*: an instruction like `ADD R1, R2, R3` needs a register file, an ALU, and a path connecting them so that operands can travel to the ALU and results can travel back.

5.3.1 The CPU's Internal Building Blocks

Before naming any particular datapath, it helps to see the pieces to be wired together. A CPU's internals can be sketched as a control unit, an ALU, and a register file, all tapping into a short internal bus, with the memory-address and memory-data registers (MAR/MDR) sitting on the far side of that bus as the gateway to main memory. The register file, the ALU, and the buses connecting them are wired together in two different ways in this chapter (Sections 5.4 and 5.5), and making that wiring, and its cost, precise is the point of what follows.

5.3.2 Register Transfer Language (RTL)

Before tracing hardware, a precise notation for describing data movement is needed. Register transfer language (RTL) writes $R1 \leftarrow [R2]$ to mean “the contents of $R2$ are transferred into $R1$.” The bracket notation $[X]$ always means *the contents of* X , whether X is a register or a memory cell, and $M[EA]$ means the memory cell at address EA . A **control step**, written $T_1, T_2, \dots$, is one clock cycle's worth of RTL transfers. This notation carries the rest of the chapter, and returns in Week 6 when the control unit that generates the $T_1, T_2, \dots$ sequence automatically, rather than by hand, is built.

5.3.3 Design Choice: One Bus or Many?

ADD R1, R2, R3 needs two register values to reach the ALU simultaneously, and one result to return. If every register and the ALU share a single physical path, only one value can be on that path in a given clock cycle — so two register values *cannot* reach the ALU in the same cycle, and the instruction needs multiple clock cycles just to move data around, before the ALU has even computed anything. If instead the ALU is given two dedicated input paths, an instruction can complete all of its data movement within a single cycle, at the cost of substantially more wiring. Two architectural choices dominate this design space: **single-bus organization** (Section 5.4), in which one shared path serves everyone and every transfer takes its turn, and **multiple-bus organization** (Section 5.5), in which several parallel paths let more happen within a single clock cycle. Building and counting both designs occupies the rest of this chapter.

5.4 Single-Bus Organization

5.4.1 The Datapath

The single-bus datapath connects the program counter (PC), memory address register (MAR), instruction register (IR), and the register file R0 . . R3 to one shared common bus. Two additional registers sit off the bus around the ALU: Y, which buffers the ALU's first operand, and Z, which buffers the ALU's result before it is written back onto the bus. The core constraint governing this whole design is that only one value can occupy the bus at any instant, so every register-to-register transfer happens through the bus, one at a time (Figure 5.1).

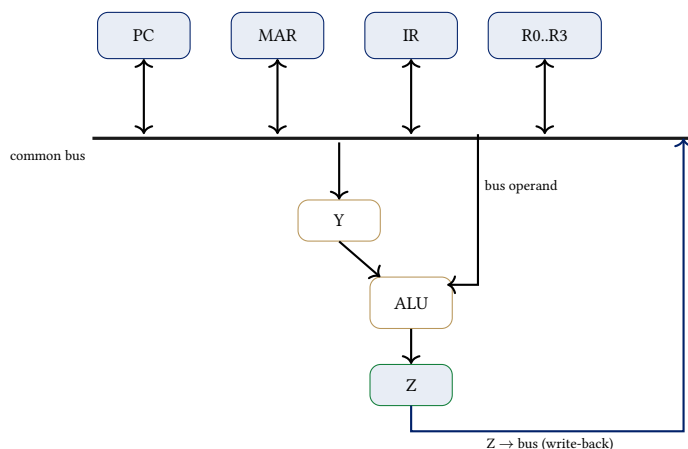


Figure 5.1: The single-bus CPU datapath — cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-4, p.129 (registers and memory on the common bus system); the Y/Z buffer registers shown here are this course's pedagogical simplification of the ALU operand path [2].

Because the ALU needs two operands but the bus can deliver only one value per cycle, one operand must be parked somewhere while the second arrives — that parking spot is register Y. Likewise, the ALU's result cannot be written back onto the bus the instant it is computed, because the bus is still occupied carrying the second operand at that moment; register Z holds the result until the bus is free. The direct consequence is that any two-operand operation on a single-bus datapath takes multiple control steps rather than one, and needs this extra buffering hardware to do it.

Example 5.4.1 ($R1 \leftarrow R2 + R3$ on a Single Bus).

Step	RTL	On the bus	What happens
T_1	$Y \leftarrow [R2]$	$[R2]$	$R2$ onto bus, latched into Y
T_2	$Z \leftarrow [Y] + [R3]$	$[R3]$	$R3$ onto bus $\rightarrow$ ALU; ALU adds Y ; result $\rightarrow Z$
T_3	$R1 \leftarrow [Z]$	$[Z]$	Z onto bus, latched into $R1$

This trace takes **3 control steps** for one **ADD** — the baseline the three-bus design in Section 5.5 is compared against. Y exists purely because the bus can carry only one operand per cycle while the ALU needs two simultaneously — Y remembers the first operand while the second is still in transit. Z exists because the ALU’s result cannot be written back to the bus in the same cycle the bus is occupied by the second operand — Z holds the result for the one extra step that requires. Both registers are the direct hardware cost of sharing a single bus: every two-operand instruction pays for this buffering.

5.4.2 Trade-offs

A single bus is cheap: one set of wires and multiplexers is the least expensive datapath to build and the easiest to extend with new registers, and its control logic is simple and uniform because every transfer looks the same. The cost of that simplicity is that every multi-operand instruction takes several control steps, so throughput is lower, and the extra Y/Z buffering hardware is unavoidable around the ALU. If every bus transfer takes exactly one clock cycle, the $R1 \leftarrow R2 + R3$ trace above takes exactly 3 cycles: one to stage the first operand, one to stage the second operand and compute, and one to write the result back.

5.5 Multiple-Bus Organization

5.5.1 The Three-Bus Datapath

The alternative design gives the register file two read ports and one write port, and connects them to the ALU through three separate physical buses: Bus A and Bus B each carry one register’s contents to a separate ALU input via a MUX (Mano’s SELA/SELB selection fields), and Bus C carries the ALU’s output directly back to the register file, while a separate decoder (SELD) independently drives the destination register’s load-enable line — the decoder selects *which* register accepts the write-back, but the data itself travels straight from the ALU to the register file, not through the decoder (Figure 5.2). Because Bus A and Bus B are separate physical paths, the register file can drive two *different* registers’ contents onto them in the very same cycle.

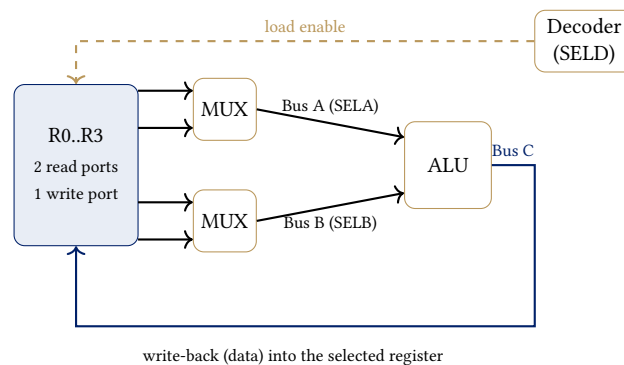


Figure 5.2: The three-bus CPU datapath — cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 8-2, p.243 (“Registers with common ALU”); the same three-bus principle is also presented in Hamacher, *Computer Organization*, Ch. 7 [2, 1].

Example 5.5.1 ($R1 \leftarrow R2 + R3$ on Three Buses).

Step	RTL	On the buses	What happens
T_1	$R1 \leftarrow [R2] + [R3]$	$A = [R2], B = [R3],$ $C = \text{result}$	Both operands read and the ALU output written back, all in one cycle

This is **1 control step** for the identical *ADD* that took 3 steps on the single-bus organization — the same instruction, the same ALU, but different wiring. The ALU no longer needs a *Y* buffer at all, because both of its inputs arrive in the same cycle directly from the register file, and Bus *C* lets the result be written back in that same cycle precisely because it is not competing with Bus *A* or Bus *B* for the same wire. The sequential bottleneck that forced the single-bus design into three steps simply does not exist here.

5.5.2 Trade-offs

Most two-operand register instructions now complete their data movement in a single control step, and no *Y/Z* buffering is needed for the common case. The cost is three times the bus wiring, plus the multiplexers needed to route any register onto any of the three buses, and control logic that must correctly drive three buses every cycle rather than one.

5.5.3 Single-Bus versus Three-Bus: A Comparison

	Single bus	Three bus
Wiring cost	Low (one bus)	High (three buses + muxes)
Cycles for $R1 \leftarrow R2 + R3$	3 (T_1, T_2, T_3)	1 (T_1)
Extra registers needed	<i>Y, Z</i> (buffering)	None (common case)
Control logic	Simple, uniform	More complex, more signals
Best fit	Cost-sensitive designs	Performance-sensitive designs

Neither design is “correct” in an absolute sense — they sit at different points on the same cost/speed trade-off curve.

5.5.4 Design-Space Limits: Why Not Ten Buses?

If three buses beat one, why not build a CPU with ten, or a hundred? Every additional bus needs its own wiring and its own multiplexer logic at every register, so cost grows with the number of buses rather than being free. Beyond that, most instructions need only two operand reads and one result write, so past three buses there is little left to parallelize *within a single instruction*. Real speedups beyond this point come from a different idea entirely: overlapping *different* instructions in time, which is exactly what pipelining (Week 11) does.

5.6 Synthesis and Summary

5.6.1 Interaction Between Addressing Mode and Bus Organization

Addressing modes carry a bus-organization cost of their own. Recall indirect addressing’s two memory reads to resolve *EA*: on a single-bus CPU, each of those reads is itself another multi-step bus transaction, so the “expensive” addressing mode and the “expensive” bus organization compound rather than simply

adding. Cheap addressing (immediate, register) paired with a cheap bus (single) yields minimal hardware but more cycles per instruction; expensive addressing (indirect) paired with an expensive bus (three-bus) can still be fast, but every design choice is paid for somewhere, in wires or in cycles, never for free.

5.6.2 Summary

An **addressing mode** is the rule for computing EA from an instruction's address field; each mode trades flexibility against extra memory or bus reads (Section 5.2). **RTL notation** ($R \leftarrow [\cdot], M[EA]$, control steps $T_1, T_2, \dots$) is how hardware-level data movement is described precisely. A **single-bus organization** is cheap to wire, moves data sequentially, needs Y/Z buffering, and costs more cycles per instruction. A **multiple-bus organization** allows parallel register reads and writes, costs fewer cycles per instruction, but needs more wiring and more complex control. The same instruction, on the same ALU, with different wiring, takes 3 cycles versus 1 — the cost/speed trade-off, made concrete. This chapter built seven addressing modes and two competing bus organizations, but in every worked trace T_1, T_2, T_3 were written by hand; Chapter 6 builds the control unit that generates that sequence automatically, for every instruction, every time it runs.

References

- [1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.
- [2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.